

Object Oriented Programming with C++

Tutorial 3

Recap : Introduction to Object-Oriented Programming

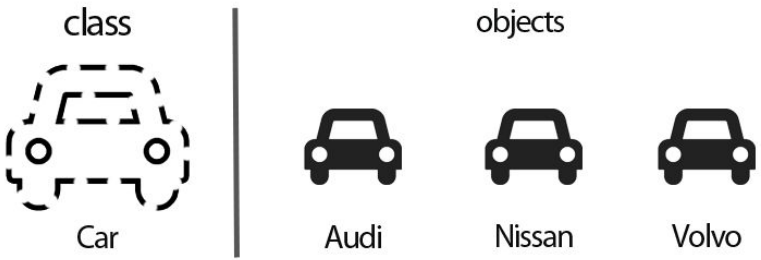
What is OOP?

Object-Oriented Programming is a programming paradigm based on the concept of "objects" that contain data and functions.

Organizes software design around objects rather than functions and logic

Enables modular, reusable, and maintainable code

Represents real-world entities as software objects

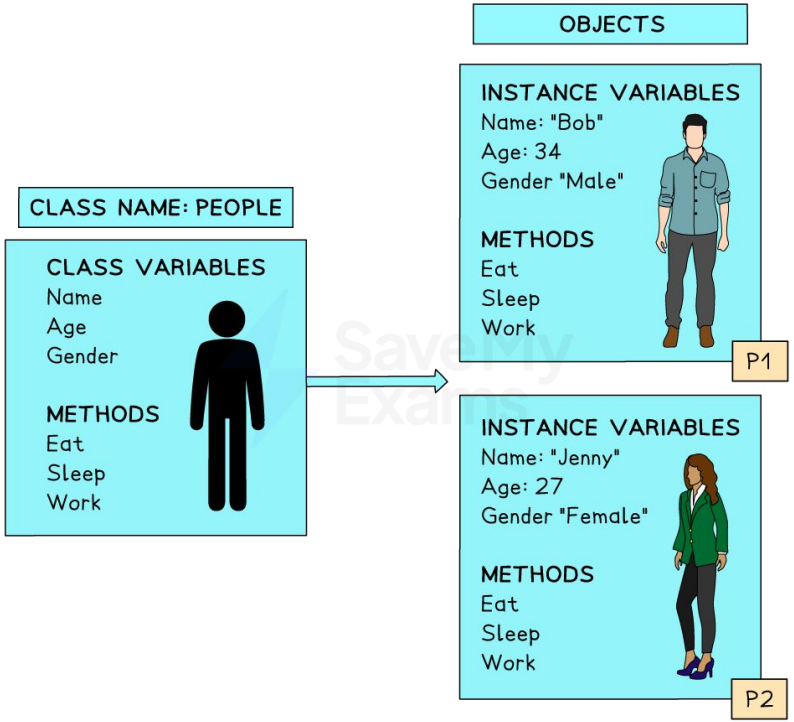


Key OOP Concepts

- **Encapsulation:** Bundling data and methods that operate on that data
- **Inheritance:** A new class of object derives properties and characteristics from another class
- **Polymorphism:** Objects can take different forms depending on context
- **Abstraction :** Hiding complex implementation details

Recap : Class and Objects

Class and Objects



Copyright © Save My Exams. All Rights Reserved

```
};
```

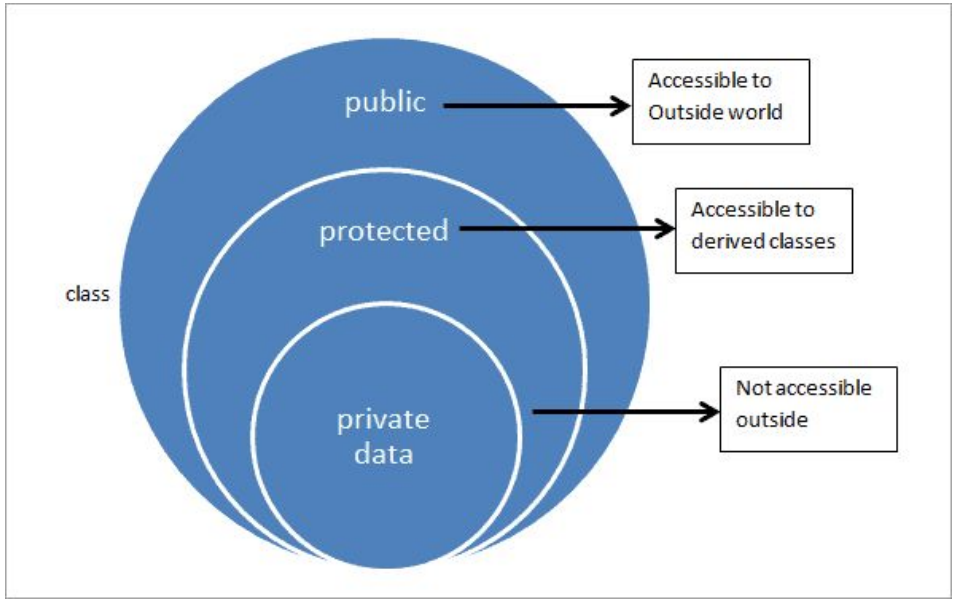
Recap : Access Specifiers

Access Specifiers in C++

Access specifiers define the accessibility of class members:

- **public:** Accessible from anywhere
- **private:** Accessible only within the class
- **protected:** Accessible within the class and derived classes

Access specifiers are fundamental to encapsulation, one of the core principles of OOP.



Best Practices

- Keep data members private
- Provide public methods to access and modify private data
- Use protected for members that derived classes need access to

Recap : Constructors and Destructors

```cpp
class Car {
private:
    string brand;
    int price;

public:
    // Default Constructor
    Car() {
        cout << "Default constructor called. Brand: " << brand << endl;
    }

    // Parameterized Constructor 1
    Car(string b) {
        brand = b;
        cout << "Parameterized constructor 1 called. Brand: " << brand << endl;
    }

    // Parameterized Constructor 2
    Car(string b, int p) {
        brand = b;
        price = p;
        cout << "Parameterized constructor 2 called. Brand: " << brand << " Price: " << price << endl;
    }

    // Copy Constructor
    Car(const Car &c) {
        brand = c.brand;
        cout << "Copy constructor called. Brand copied: " << brand << endl;
    }

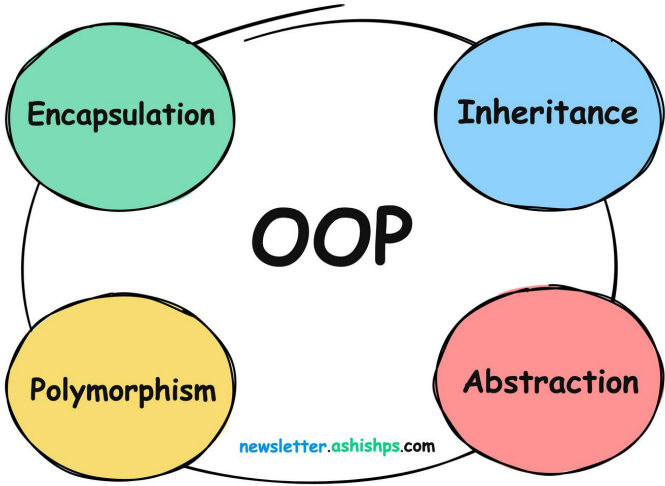
    // Destructor
    ~Car() {
        cout << "Destructor called for brand: " << brand << endl;
    }
};
```

```cpp
int main() {
    Car car1;               // Default constructor
    Car car2("BMW");        // Parameterized constructor 1
    Car car3("BMW", 1000);  // Parameterized constructor 2
    Car car4 = car2;        // Copy constructor

    return 0;
}
```

Core Concepts of OOP

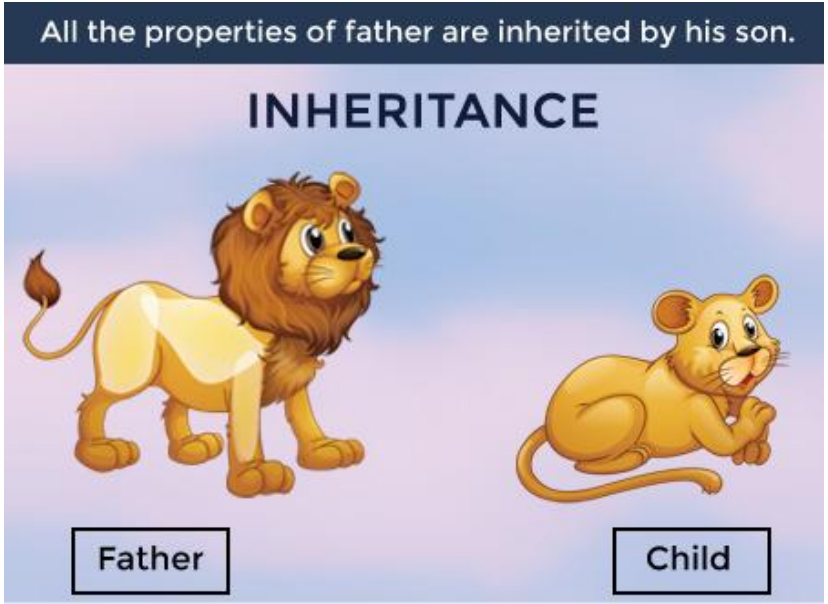
- **Inheritance:**
  - The mechanism by which one class acquires the properties and behaviors of another.
- **Polymorphism:**
  - The ability of an object to take on many forms.
- **Encapsulation:**
  - Bundling data and methods that operate on the data into a single unit (class
- **Abstraction:**
  - Hiding complex implementation details and showing only essential features.



Inheritance

What is Inheritance?

- Inheritance is a fundamental principle of OOP that allows a new class (the subclass or derived class) to be based on an existing class (the superclass or base class).
- **Key Idea**: The subclass inherits attributes and methods from the superclass.
- **Main Benefit**: It promotes **code reusability**. You write common code once in a base class and reuse it across multiple specialized subclasses.
- **Relationship**: It establishes an "**is-a**" relationship.
    - For example, a Car is-a Vehicle.
    - Lion is-an Animal



Inheritance

Base Class and Derived Class

- **Base Class / Parent Class** – The class being inherited from.
- **Derived Class / Child Class** – The class that inherits from the base class.

```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    void eat() { cout << "Eating...\n"; }
};

class Dog : public Animal {
public:
    void bark() { cout << "Barking...\n"; }
};

int main() {
    Dog d;
    d.eat();   // Inherited from Animal
    d.bark();  // Dog's own function
}
```

Inheritance

Modes of Inheritance

- **Public Inheritance**: The most common mode. Public members of the base class stay public.
- **Protected Inheritance**: Public members of the base class become protected in the subclass.
- **Private Inheritance**: All members of the base class become private in the subclass.

```
class A
{
public:
    int x;
protected:
    int y;
private:
    int z;
};

class B : public A
{
    // x is public
    // y is protected
    // z is not accessible from B
};

class C : protected A
{
    // x is protected
    // y is protected
    // z is not accessible from C
};

class D : private A    // 'private' is default for classes
{
    // x is private
    // y is private
    // z is not accessible from D
};
```

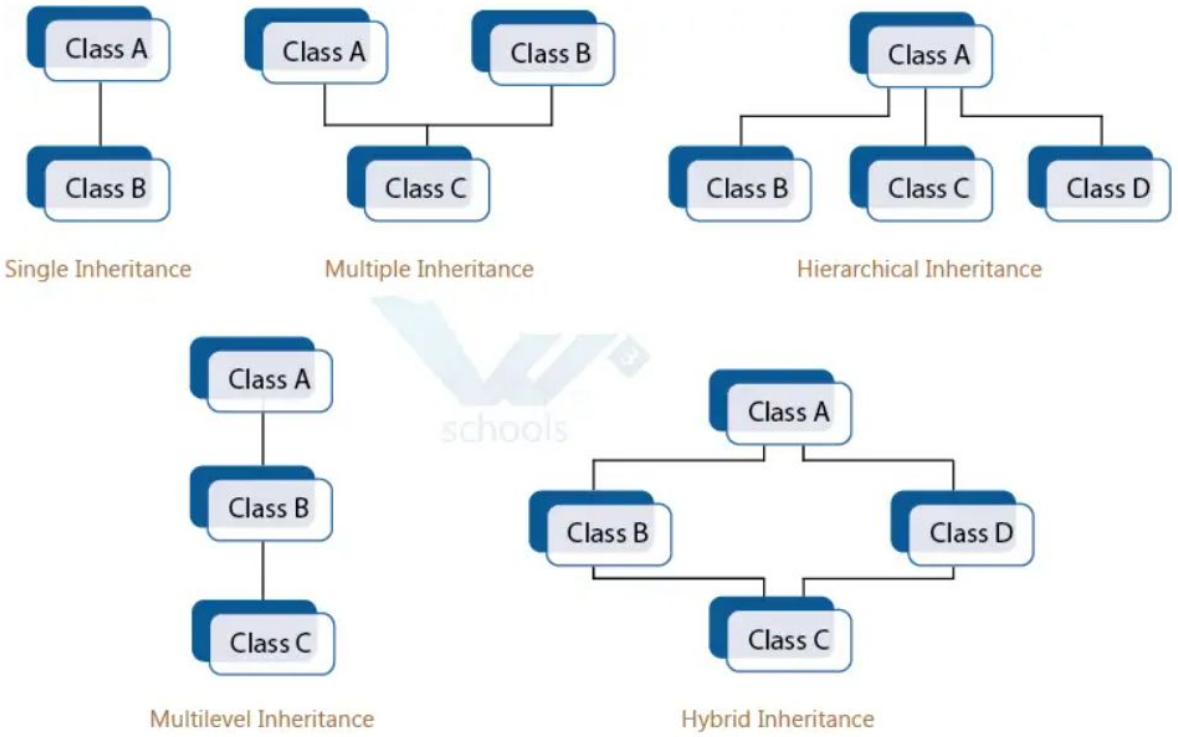
Inheritance

Modes of Inheritance

Base Class Member	public Inheritance	protected Inheritance	private Inheritance
public	public	protected	private
protected	protected	protected	private
private	Not Accessible	Not Accessible	Not Accessible

Inheritance

Types of Inheritance



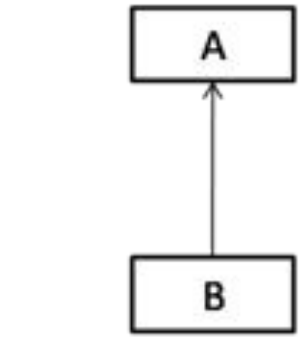
Single Inheritance

One base class → One derived class

```cpp
class Animal {
public:
    void breathe() { cout << "Breathing...\n"; }
};

class Dog : public Animal {
public:
    void bark() { cout << "Barking...\n"; }
};

int main() {
    Dog d;
    d.breathe();  // from Animal
    d.bark();     // from Dog
}
```



Simple Inheritance

Multiple Inheritance

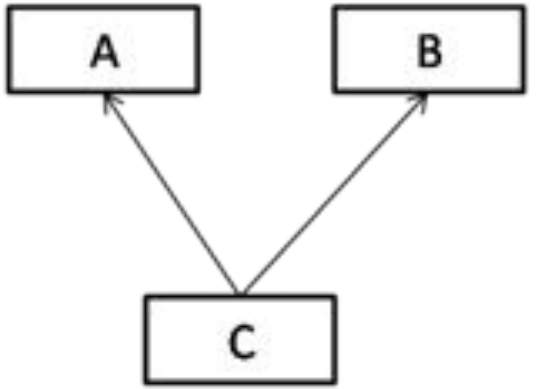
One derived class inherits from multiple base classes

```cpp
class Animal {
public:
    void eat() { cout << "Eating...\n"; }
};

class Pet {
public:
    void play() { cout << "Playing...\n"; }
};

class Dog : public Animal, public Pet {
public:
    void bark() { cout << "Barking...\n"; }
};

int main() {
    Dog d;
    d.eat();
    d.play();
    d.bark();
}
```



Multiple Inheritance

Multilevel Inheritance

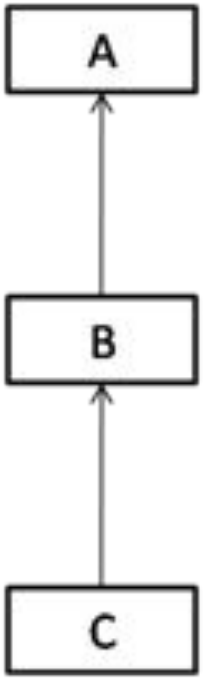
A derived class becomes the base for another class (chain)

```
class Animal {
public:
    void eat() { cout << "Eating...\n"; }
};

class Mammal : public Animal {
public:
    void walk() { cout << "Walking...\n"; }
};

class Dog : public Mammal {
public:
    void bark() { cout << "Barking...\n"; }
};

int main() {
    Dog d;
    d.eat();
    d.walk();
    d.bark();
}
```



Hierarchical Inheritance

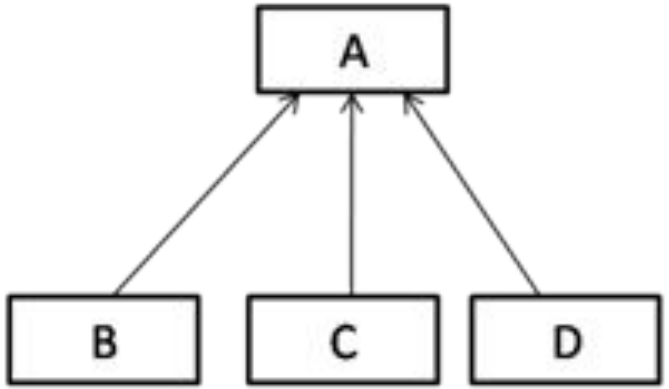
One base class → multiple derived classes

```cpp
class Animal {
public:
    void eat() { cout << "Eating...\n"; }
};

class Dog : public Animal {
public:
    void bark() { cout << "Barking...\n"; }
};

class Cat : public Animal {
public:
    void meow() { cout << "Meowing...\n"; }
};

int main() {
    Dog d;
    Cat c;
    d.eat(); d.bark();
    c.eat(); c.meow();
}
```



Hierarchical Inheritance

Hybrid Inheritance

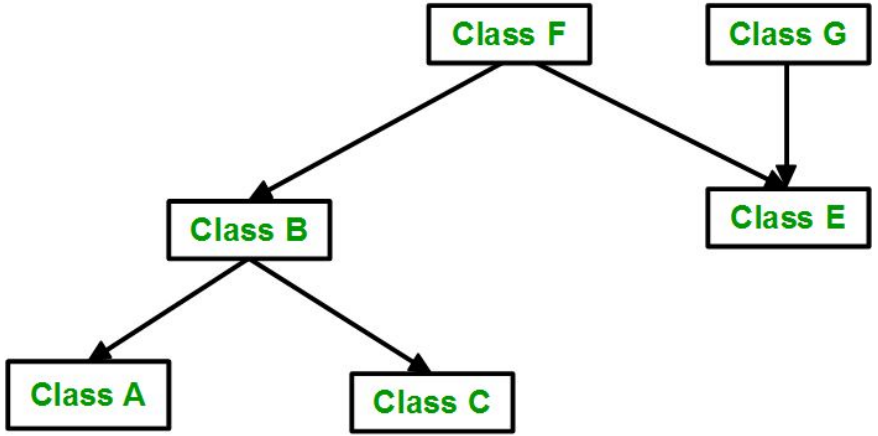
A combination of two or more inheritance types

```
class Animal {
public:
    void eat() { cout << "Eating...\n"; }
};

class Mammal : public Animal {
public:
    void walk() { cout << "Walking...\n"; }
};

class Pet {
public:
    void play() { cout << "Playing...\n"; }
};

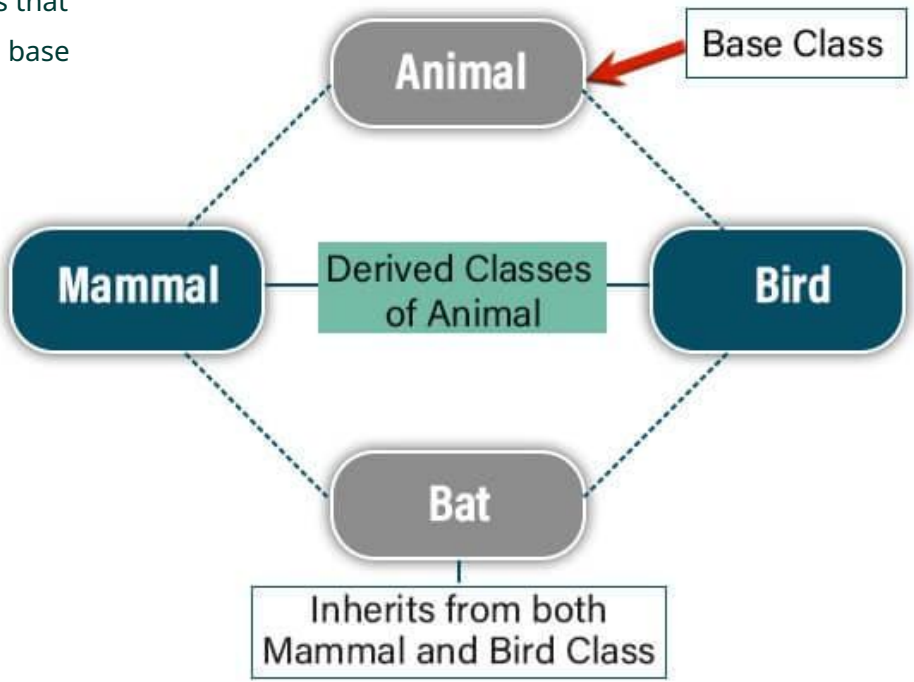
class Dog : public Mammal, public Pet {
public:
    void bark() { cout << "Barking...\n"; }
};
```



The Diamond Problem

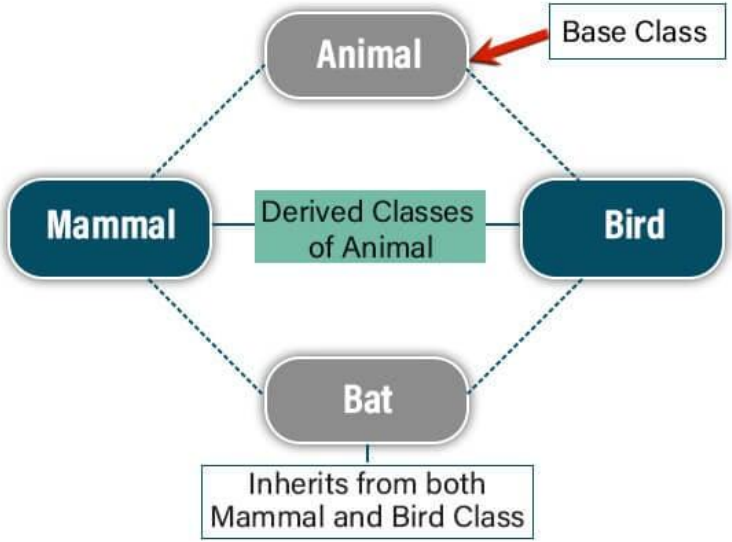
A common issue with multiple inheritance is the "Diamond Problem."

- Occurs when a derived class inherits from two classes that share a common base, causing multiple copies of the base class.



The Diamond Problem

The compiler sees two versions of eat() (one from Mammal::Animal, another from Bird::Animal) and doesn't know which one to call - Ambiguity




```
class Animal {
public:
    void eat() { cout << "Animal is eating...\n"; }
};

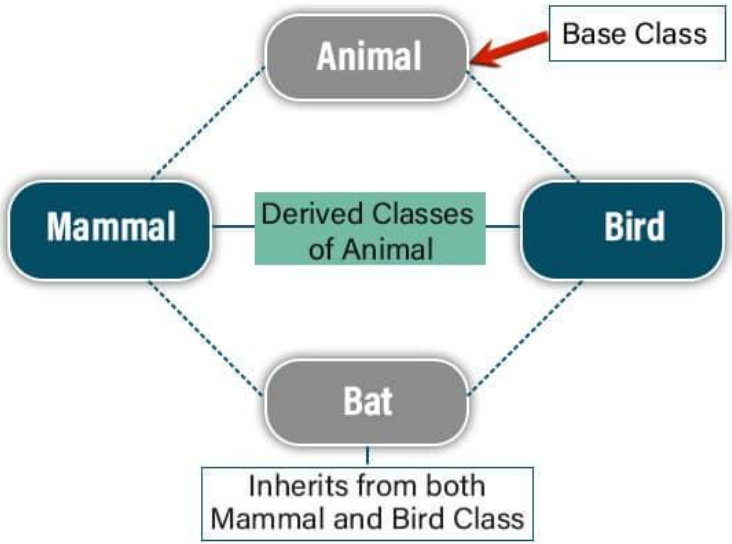
class Mammal : public Animal { };
class Bird   : public Animal { };

class Bat : public Mammal, public Bird { };

int main() {
    Bat b;
    // b.eat(); ❌ Ambiguity: compiler error
```

The Diamond Problem

Solution 1: Scope Resolution Operator (::)




```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    void eat() { cout << "Animal is eating...\n"; }
};

class Mammal : public Animal { };
class Bird   : public Animal { };

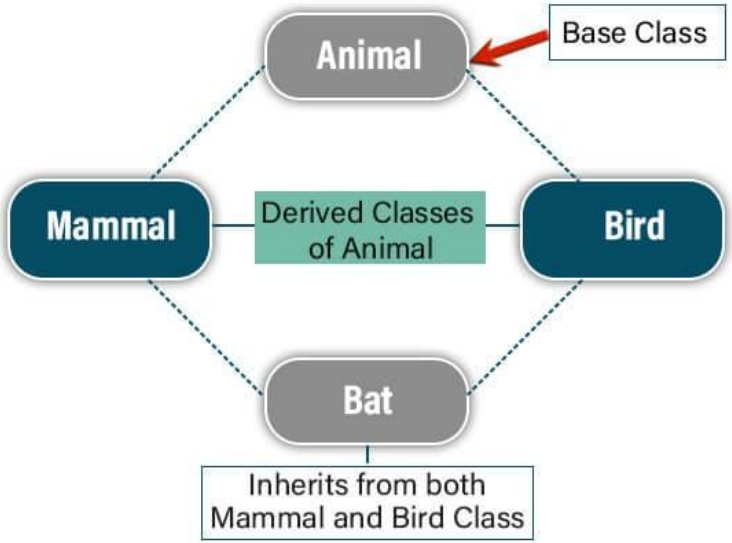
class Bat : public Mammal, public Bird { };

int main() {
    Bat b;
    // b.eat();  // ❌ Error: ambiguous (two copies of Animal)

    b.Mammal::eat(); // ✅ Resolves ambiguity
    b.Bird::eat();   // ✅ Resolves ambiguity
}
```

The Diamond Problem

Solution 2: Use virtual inheritance so that only one copy of the base class exists.




```cpp
class Animal {
public:
    void eat() { cout << "Animal is eating...\n"; }
};

class Mammal : virtual public Animal { };
class Bird   : virtual public Animal { };

class Bat : public Mammal, public Bird { };

int main() {
    Bat b;
    b.eat(); // ✅ No ambiguity (only one Animal copy)
}
```

Summary - Inheritance

Key Points

- Inheritance is a powerful feature for creating l**ogical hierarchies** and **reusing code**.
- It establishes an **"is-a"** relationship between classes.
- C++ supports various types of inheritance, including single, multiple, and multilevel.
- Be aware of the Diamond Problem with multiple inheritance and know how to solve it using **virtual inheritance** or the **scope resolution operator.**

Polymorphism

What is Polymorphism?

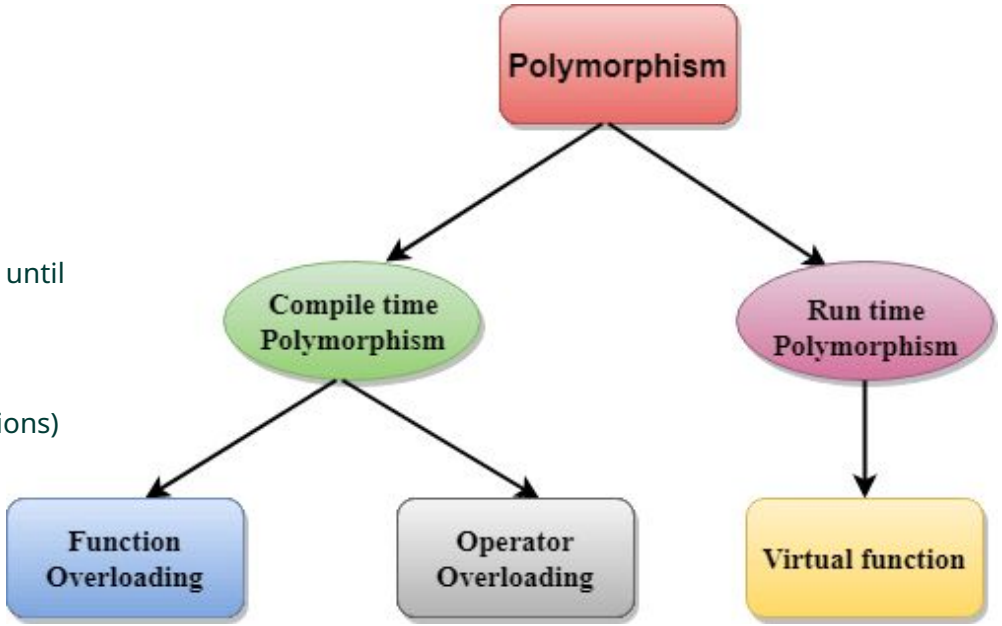
- The word Polymorphism comes from Greek words:
  - **Poly** -> many
  - **Morph** -> forms
- In simple terms, polymorphism is the ability of a single interface (like a function or an object) to have multiple forms or behaviors.



Polymorphism

Types of Polymorphism in C++

- Compile-Time Polymorphism (Static Binding):
  - The decision of which function to call is made at compile time.
  - It is achieved through:
    - Function Overloading
    - Operator Overloading
- Runtime Polymorphism (Dynamic Binding):
  - The decision of which function to call is delayed until runtime.
  - It is achieved through:
    - Function Overriding (using Virtual Functions)



Function Overloading

What is Function Overloading?

- Function overloading allows multiple functions with the **same name** but **different signatures**.
- It enables compile-time polymorphism.

- Overloaded functions must differ in their **function's signature.**
- A function's signature is determined by its:
  - Function name
  - Number of parameters
  - Type of parameters
  - Order of parameters
- Functions can not be overloaded if they differ only in the **return type** or **parameter names.**

```
class Calculator {
public:
    // Add two integers
    int add(int a, int b) {
        return a + b;
    }

    // Add three integers
    int add(int a, int b, int c) {
        return a + b + c;
    }

    // Add two doubles
    double add(double a, double b) {
        return a + b;
    }
};

int main() {
    Calculator calc;

    cout << "Sum of 2 + 3 = " << calc.add(2, 3) << endl;
    cout << "Sum of 2 + 3 + 4 = " << calc.add(2, 3, 4) << endl;
    cout << "Sum of 2.5 + 3.7 = " << calc.add(2.5, 3.7) << endl;

    return 0;
}
```

Function Overloading

What is Function Overloading?

- Function overloading allows multiple functions with the **same name** but **different signatures**.
- It enables compile-time polymorphism.

- Overloaded functions must differ in their **function's signature.**
- A function's signature is determined by its:
  - Function name
  - Number of parameters
  - Type of parameters
  - Order of parameters
- Functions can not be overloaded if they differ only in the **return type** or **parameter names.**

```
class Test {
public:
    int getValue() {
        return 10;
    }

    double getValue() {   // ❌ Error: only return type differs
        return 10.5;
    }
};
```

Operator Overloading

What is Operator Overloading?

- Allows custom behavior for operators when used with user-defined types (e.g., classes).
- Improves code readability and intuitive use of objects.

```cpp
#include <iostream>
using namespace std;

class Point {
    int x, y;

public:
    Point(int xVal, int yVal) : x(xVal), y(yVal) {}

    // Overload the + operator
    Point operator + (const Point& other) {
        return Point(x + other.x, y + other.y);
    }

    void display() {
        cout << "(" << x << ", " << y << ")" << endl;
    }
};

int main() {
    Point p1(2, 3);
    Point p2(4, 5);

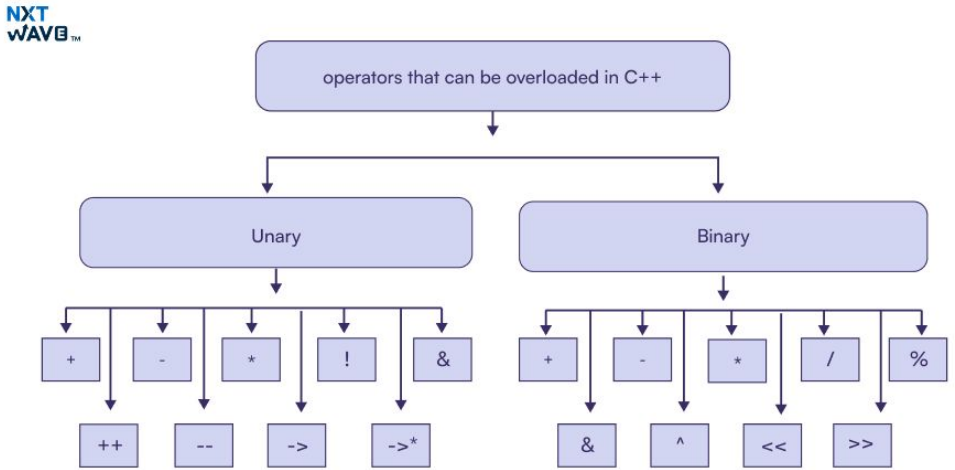
    Point p3 = p1 + p2;  // operator+ is called
    p3.display();        // Output: (6, 8)

    return 0;
}
```

Operator Overloading

Operators that can NOT be Overloaded

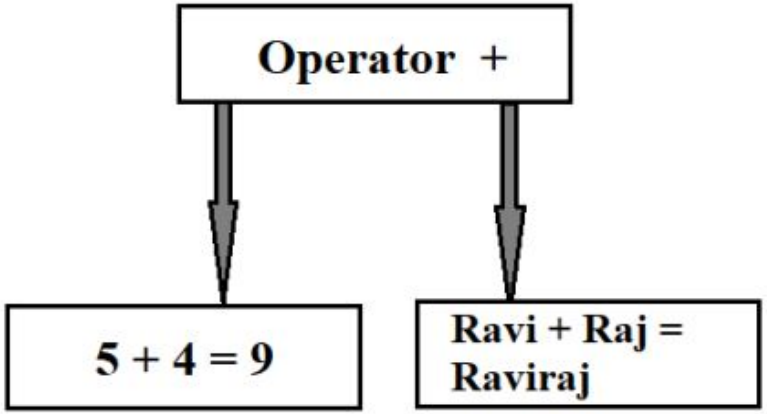
- The dot operator (.)
- The scope resolution operator (::)
- The dereferencing operator (*)
- The ternary conditional operator (? :)
- The sizeof operator



Operator Overloading

Limitations of operator overloading

- Some operators are not overloadable
- Cannot Change Precedence or Associativity
- Cannot Create New Operators
- Overuse Can Reduce Readability
- At Least One Operand Must Be a User-Defined Type
    - You cannot overload an operator for two primitive types like int + float.



Operator Overloading

Approaches for operator overloading

- Overloading unary operator.
- Overloading binary operator.
- Overloading operators using a friend function

```cpp
#include <iostream>
using namespace std;

class Point {
    int x, y;

public:
    Point(int xVal, int yVal) : x(xVal), y(yVal) {}

    // Overload the + operator
    Point operator + (const Point& other) {
        return Point(x + other.x, y + other.y);
    }

    void display() {
        cout << "(" << x << ", " << y << ")" << endl;
    }
};

int main() {
    Point p1(2, 3);
    Point p2(4, 5);

    Point p3 = p1 + p2;  // operator+ is called
    p3.display();        // Output: (6, 8)

    return 0;
}
```

Operator Overloading

Overloading Unary Operator

- Operates on one operand (e.g., -obj, ++obj).
- Can be overloaded as member or friend.

```cpp
#include <iostream>
using namespace std;

class Count {
    int value;
public:
    Count(int v) : value(v) {}

    void operator++() {  // Unary operator++
        ++value;
    }

    void display() {
        cout << "Value: " << value << endl;
    }
};

int main() {
    Count c(5);
    c.display(); // Output: Value: 5
    ++c;  // Calls overloaded operator++
    c.display();  // Output: Value: 6
    return 0;
}
```

Operator Overloading

Overloading Binary Operator

- Operates on two operands (e.g., obj1 + obj2).
- Often overloaded as member or friend.

```cpp
#include <iostream>
using namespace std;

class Point {
    int x;
public:
    Point(int val) : x(val) {}

    Point operator+(const Point& p) {  // Binary operator+
        return Point(x + p.x);
    }

    void display() {
        cout << "X: " << x << endl;
    }
};

int main() {
    Point p1(3), p2(7);
    Point p3 = p1 + p2;  // Calls operator+
    p3.display();        // Output: X: 10
    return 0;
}
```

Operator Overloading

Overloading Using Friend Function

- Allows operator function to access private members.
- Useful when left operand isn't a class object.

```cpp
#include <iostream>
using namespace std;

class Distance {
    int meters;
public:
    Distance(int m) : meters(m) {}

    // Declare friend function
    friend Distance operator+(const Distance&, const Distance&);

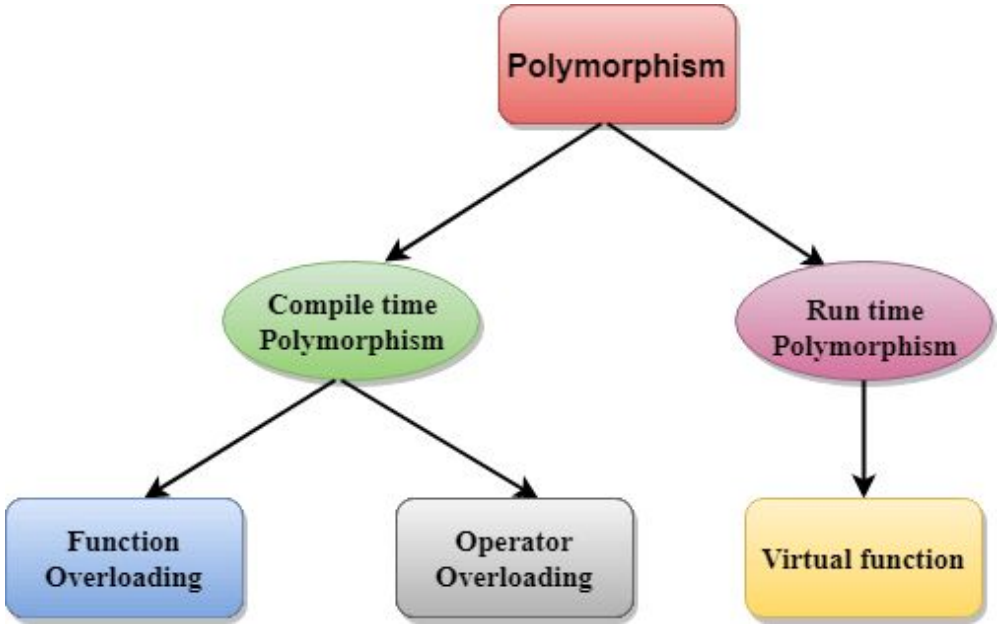
    void display() {
        cout << "Meters: " << meters << endl;
    }
};

// Define friend function
Distance operator+(const Distance& d1, const Distance& d2) {
    return Distance(d1.meters + d2.meters);
}

int main() {
    Distance d1(20), d2(30);
    Distance d3 = d1 + d2;  // Calls friend operator+
    d3.display();           // Output: Meters: 50
    return 0;
}
```

Polymorphism

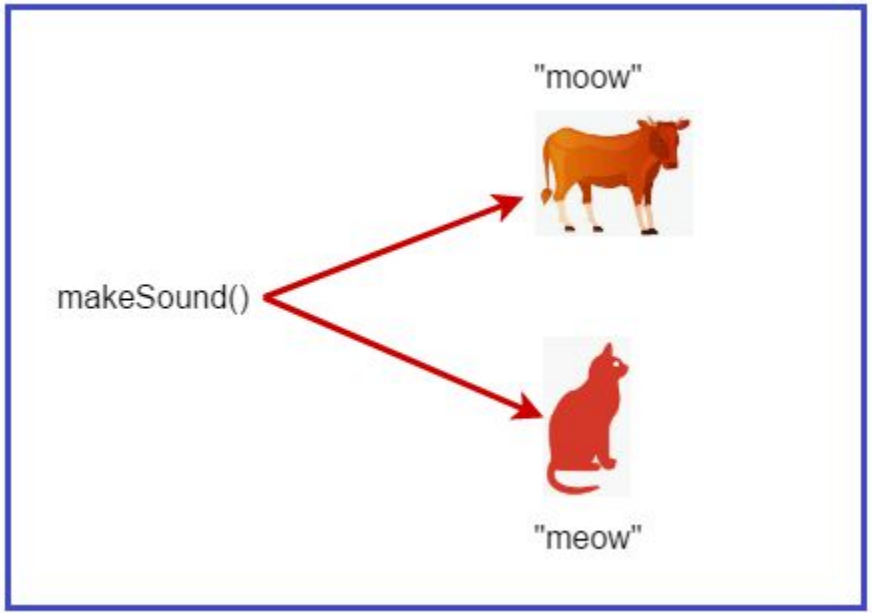
Types of Polymorphism in C++



Function Overriding

What is Function Overriding?

- When a derived (child) class provides its own version of a function that is already defined in the base (parent) class.
- The same function name and parameters are used.
- Decided at runtime → this is runtime polymorphism.



Function Overriding

What is Function Overriding?

- When a derived (child) class provides its own version of a function that is already defined in the base (parent) class.
- The same function name and parameters are used.
- Decided at runtime → this is runtime polymorphism.

```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    void makeSound() {   // Base class version
        cout << "Some generic animal sound\n";
    }
};

class Cow : public Animal {
public:
    void makeSound() {   // Reimplement Animal's function
        cout << "Moow\n";
    }
};

class Cat : public Animal {
public:
    void makeSound() {   // Reimplement Animal's function
        cout << "Meow\n";
    }
};

int main() {
    Animal a;
    Cow c;
    Cat k;

    a.makeSound();   // ?? Calls Animal's version "Some generic animal sound"
    c.makeSound();   // ?? Calls Cows's version "Moow"
    k.makeSound();   // ?? Calls Cat's version "Meow"

    return 0;
}
```

Function Overriding

Virtual Functions

- A virtual function is a member function in a base class that you expect to redefine in derived classes.
- You declare it by using the virtual keyword in the base class.
- Why are they important?
    - When you call a virtual function through a base class pointer or reference, C++ will call the correct overridden version from the derived class. This is called dynamic dispatch.

```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    // Declare as virtual
    virtual void makeSound() {
        cout << "Some generic animal sound\n";
    }
};

class Cow : public Animal {
public:
    void makeSound() override {   // Overrides Animal's function
        cout << "Moow\n";
    }
};

class Cat : public Animal {
public:
    void makeSound() override {   // Overrides Animal's function
        cout << "Meow\n";
    }
};

int main() {
    Animal a;

    Cow c;
    Cat k;

    a.makeSound();   // ?? Calls Animal's version "Some generic animal sound"
    c.makeSound();   // ?? Calls Cows's version "Moow"
    k.makeSound();   // ?? Calls Cat's version "Meow"

    return 0;
}
```

Function Overriding

Non-virtual

- Even though ‘a’ points to Cow or Cat, the base class function runs.

```
4  class Animal {
5  public:
6      void makeSound() {
7          cout << "Some generic animal sound\n";
8      }
9  };
10
11 class Cow : public Animal {
12 public:
13     void makeSound() { cout << "Moow\n"; }
14 };
15
16 class Cat : public Animal {
17 public:
18     void makeSound() { cout << "Meow\n"; }
19 };
20
21 int main() {
22     Animal* a;
23     Cow c;
24     Cat k;
25
26     a = &c;
27     a->makeSound();  // ? Calls Animal's version
28     a = &k;
29     a->makeSound();  // ? Calls Animal's version
30 }
```

```
Some generic animal sound
Some generic animal sound
```

Virtual

- Now the correct derived function executes.

```
4  class Animal {
5  public:
6      virtual void makeSound() {
7          cout << "Some generic animal sound\n";
8      }
9  };
10
11 class Cow : public Animal {
12 public:
13     void makeSound() override { cout << "Moow\n"; }
14 };
15
16 class Cat : public Animal {
17 public:
18     void makeSound() override { cout << "Meow\n"; }
19 };
20
21 int main() {
22     Animal* a;
23     Cow c;
24     Cat k;
25
26     a = &c;
27     a->makeSound();  // ? "Moow"
28     a = &k;
29     a->makeSound();  // ? "Meow"
30 }
```

```
Moow
Meow
```

Function Overriding

Pure Virtual Functions

- A virtual function with **no implementation** in the base class.
- Declared using = 0.
- Makes a class **abstract** (cannot create objects of it).
- Must be **overridden** in derived classes.

```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void makeSound() = 0;
};

class Cow : public Animal {
public:
    void makeSound() override { cout << "Moow\n"; }
};

class Cat : public Animal {
public:
    void makeSound() override { cout << "Meow\n"; }
};

int main() {
    // Animal a; ? Not allowed (abstract class)

    Cow c;
    Cat k;

    c.makeSound();  // "Moow"
    k.makeSound();  // "Meow"
}
```

Function Overriding

The override Keyword

- Mistakes can happen when overriding. You might misspell the function name or use the wrong parameters. The compiler wouldn't know you intended to override.
- C++11 introduced the override keyword to prevent this.
    - **Purpose**: It tells the compiler that you intend to override a base class virtual function.
    - **Benefit**: If the function in the base class doesn't exist, doesn't have the same signature, or isn't virtual, the compiler will generate an error.

```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void makeSound() = 0;
};

class Cow : public Animal {
public:
    void makeSound() override { cout << "Moow\n"; }
};

class Cat : public Animal {
public:
    void makeSound() override { cout << "Meow\n"; }
};

int main() {
    // Animal a; ? Not allowed (abstract class)

    Cow c;
    Cat k;

    c.makeSound();  // "Moow"
    k.makeSound();  // "Meow"
}
```

Summary - Polymorphism

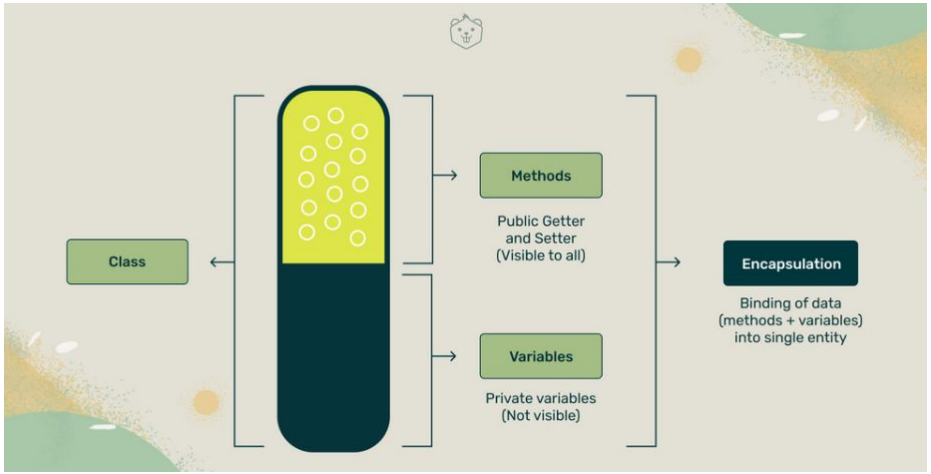
Key points

- Polymorphism allows a single interface to represent **different underlying forms**.
- Compile-Time Polymorphism is achieved via **function overloading** and **operator overloading**, It is resolved early.
- Runtime Polymorphism is achieved via **virtual functions and overriding**, allowing for flexible and dynamic code.
- Use the **override** keyword to prevent bugs.
- Use **abstract classes** and **pure virtual functions** to create interfaces and define contracts for subclasses.

Encapsulation

What is Encapsulation?

- Bundling **data (attributes)** and the **methods (functions)** that operate on that data into a **single unit**, which we call a class.
- Think of it like a capsule:
  - The capsule holds all the necessary ingredients (data and methods) together.
  - It also protects the ingredients from the outside world.



Encapsulation

What is Encapsulation?

- This protection is achieved by restricting direct access to some of the object's components using access modifiers:
  - private
  - protected
  - public
- Direct access to private members is not allowed.
- **Getter** and **Setter** methods provide safe access.
- Protects the integrity of the data.

```
class Car {
private:        // data hidden (cannot be accessed directly)
    int speed;
    int fuel;

public:         // controlled access
    void setSpeed(int s) {
        if(s >= 0 && s <= 200)   // validation (optional)
            speed = s;
    }

    int getSpeed() {
        return speed;
    }

    void setFuel(int f) {
        if(f >= 0 && f <= 100)   // validation (optional)
            fuel = f;
    }

    int getFuel() {
        return fuel;
    }
};
```

Abstraction

What is Abstraction?

- Abstraction is the concept of **hiding unnecessary details** and **showing only the essential features** of an object or system.



Real Life Example of Abstraction

Abstraction

What is Abstraction?

- Achieved using abstract classes (with **pure virtual functions**) or interfaces.
- The user knows what an object does, but not how it does it.

```
// Abstract class
class Vehicle {
public:
    // Pure virtual function (no implementation here)
    virtual void start() = 0;
    virtual void stop() = 0;
};
```

```
// Concrete class (implements abstract class)
class Car : public Vehicle {
public:
    void start() override {
        cout << "Car started with a key...\n";
    }

    void stop() override {
        cout << "Car stopped using brakes...\n";
    }
};

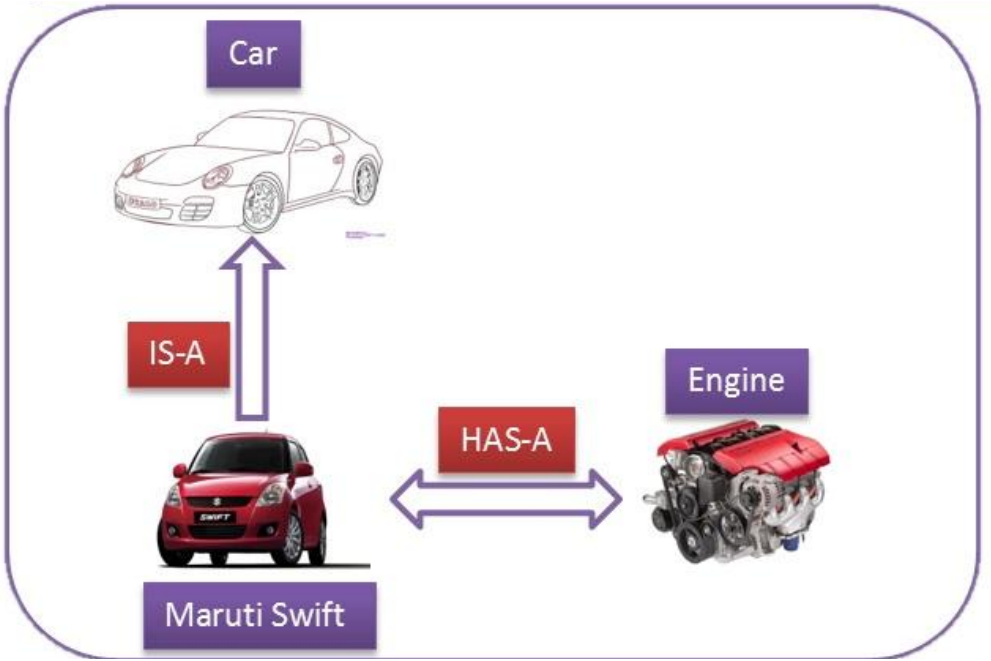
// Another concrete class
class Bike : public Vehicle {
public:
    void start() override {
        cout << "Bike started with a self button...\n";
    }

    void stop() override {
        cout << "Bike stopped using disc brakes...\n";
    }
};
```

Composition

What is Composition?

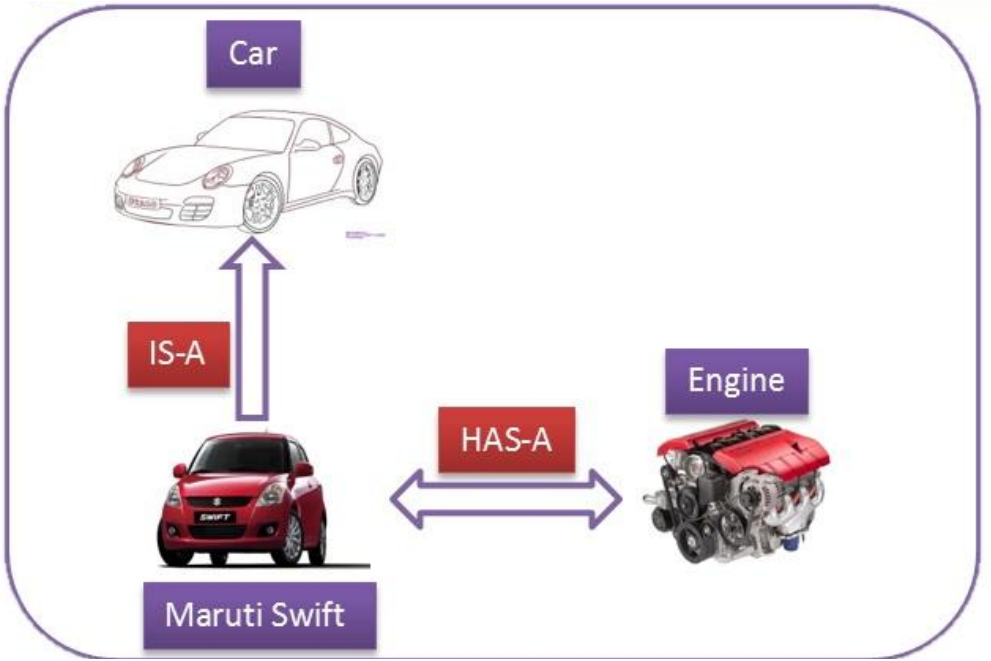
- Composition is a design principle where a class contains an object of another class as one of its members.
- It models a **"has-a"** relationship.



Composition

What is Composition?

- Composition is a design principle where a class contains an object of another class as one of its members.
- It models a **"has-a"** relationship.



Composition

What is Composition?

- The containing object (the Car) is responsible for the lifecycle of the contained object (the Engine). When the Car is destroyed, its Engine is also destroyed.

```
// Engine class
class Engine {
public:
    void start() {
        cout << "Engine started...\n";
    }
    void stop() {
        cout << "Engine stopped...\n";
    }
};

// Car class HAS an Engine
class Car {
private:
    Engine engine;   // Composition

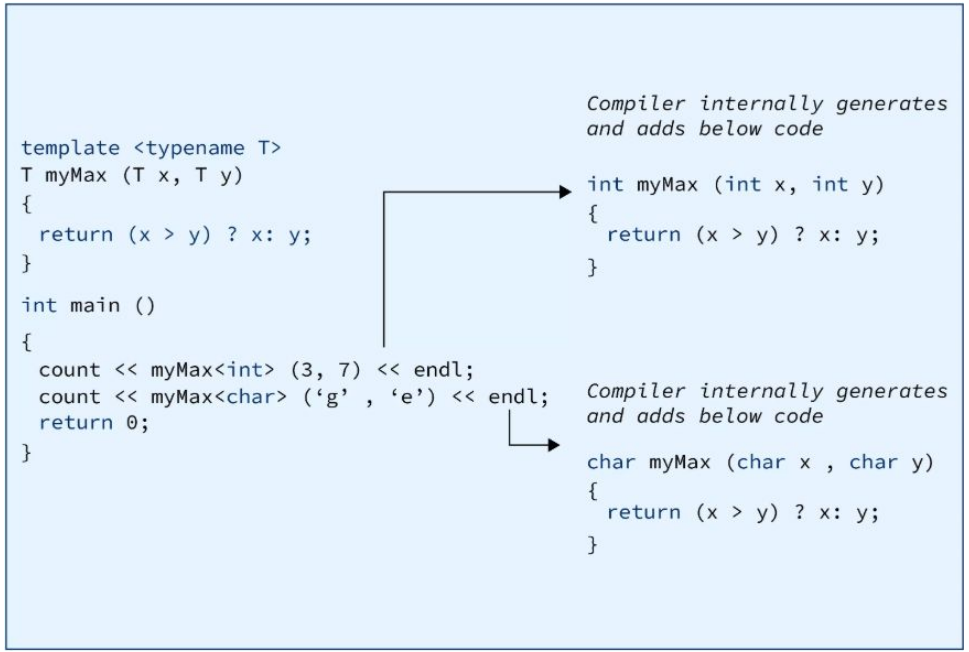
public:
    void startCar() {
        engine.start();
        cout << "Car is running...\n";
    }

    void stopCar() {
        engine.stop();
        cout << "Car has stopped.\n";
    }
};
```

Templates

What are Templates?

- Templates are a powerful C++ feature for writing generic code.
- The core idea is to pass a data type as a parameter.
- This allows you to write a single function or class that can work with different data types, instead of writing multiple overloaded versions.
- Two types:
  - Function Templates
  - Class Templates



Templates

Function Templates

- Function templates allow writing generic functions.
- Same function works with different data types without rewriting code.
- Replace type with generic type parameter (T).
- Compiler generates separate functions for each type when called.
- Avoids writing duplicate functions like getMaxInt, getMaxDouble, etc.

```cpp
#include <iostream>
using namespace std;

// Function template
template <typename T>
T getMax(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << "Max of 10 and 20: " << getMax(10, 20) << endl;         // int
    cout << "Max of 5.5 and 2.3: " << getMax(5.5, 2.3) << endl;     // double
    cout << "Max of 'A' and 'Z': " << getMax('A', 'Z') << endl;     // char

    return 0;
}
```

Templates

Function Templates with Multiple Arguments

- Function templates also allow writing generic functions with multiple arguments

```cpp
#include <iostream>
using namespace std;

// Function template with two type parameters
template <typename T1, typename T2>
void showSum(T1 a, T2 b) {
    cout << "Sum: " << a + b << endl;
}

int main() {
    // int + int
    showSum<int, int>(10, 20);

    // double + int
    showSum<double, int>(5.5, 2);

    // string + string
    showSum<string, string>("Hello ", "World");

    return 0;
}
```

Templates

Function Templates with a non-type parameter

- Non-type parameters are constants known at compile time, like integers, array sizes, or pointers

```cpp
#include <iostream>
using namespace std;

// Function template with a type parameter T and a non-type parameter N
template <typename T, int N>
void multiplyValue(T value) {
    cout << value << " multiplied by " << N << " = " << value * N << endl;
}

int main() {
    // int value, multiply by 5
    multiplyValue<int, 5>(10);

    // double value, multiply by 3
    multiplyValue<double, 3>(2.5);

    // float value, multiply by 4
    multiplyValue<float, 4>(1.2f);

    return 0;
}
```

Templates

Class Templates

- Class templates allow creating generic classes.
- Just like function templates, but for objects and data structures.

```cpp
// Class template
template <typename T>
class Box {
private:
    T value;
public:
    Box(T v) : value(v) {}

    T getValue() {
        return value;
    }
};

int main() {
    Box<int> intBox(100);
    Box<double> doubleBox(45.67);
    Box<string> stringBox("Hello");

    cout << "Int Box: " << intBox.getValue() << endl;
    cout << "Double Box: " << doubleBox.getValue() << endl;
    cout << "String Box: " << stringBox.getValue() << endl;

    return 0;
}
```

References

- https://github.com/Chamika36/IS2203-OOP-Tutorials.git
- https://www.geeksforgeeks.org/cpp/object-oriented-programming-in-cpp/